

1. Deskripsi Proyek

Kelarin adalah platform manajemen tugas akademik berbasis cloud-native yang dirancang untuk memfasilitasi kolaborasi mahasiswa secara terstruktur. Aplikasi ini mengintegrasikan:

- Penambahan tugas
- Penentuan deadline
- Distribusi tanggung jawab tim

dalam satu dashboard terpadu.

Dengan arsitektur yang *scalable* dan penyimpanan terpusat, Kelarin memungkinkan:

- Monitoring progres secara *real-time*
- Transparansi kerja tim
- Akses data dari mana saja

Tujuan utama:

- Mengatasi dokumentasi tugas yang terfragmentasi
- Menghilangkan pembagian peran yang tidak jelas
- Meningkatkan efisiensi kerja kelompok

2. Profil Tim (ETHEREAL TEAM)

Nama	NIM	Peran
Tiya Mitra Ayu	10231088	Lead Backend
Amazia Devid Saputra	10231013	Lead Frontend
Alsha Dwi Cahya	10231011	Lead Container
Andini Permata Sari	10231015	Lead QA & Docs
Ansellma Tita Pakartiwuri Putri	10231017	Lead Deploy & CI/CD

3. Tech Stack

Backend

- **Python 3.10+** → Bahasa utama
- **FastAPI** → Framework REST FastAPI
- **PostgreSQL 16** → Database relasional
- **Nginx** → API Gateway & reverse proxy

Frontend

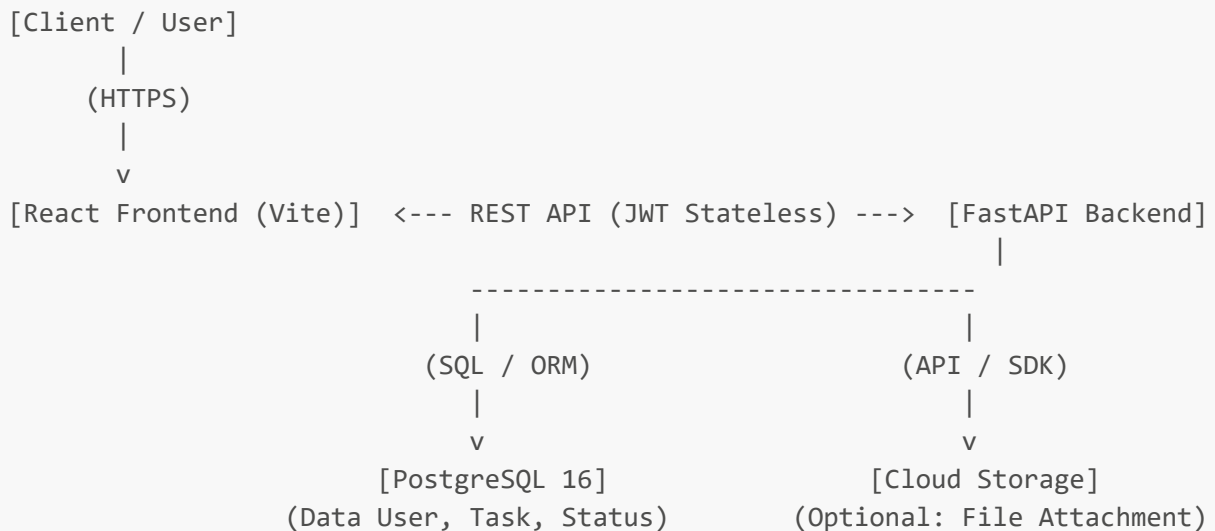
- **React.js** → UI berbasis komponen
- **Vite** → Tooling SPA modern

DevOps & Cloud

- **Docker** → Containerization
- **Docker Compose** → Orkestrasi multi-container
- **GitHub Actions** → CI/CD pipeline
- **Railway** → Hosting backend & database

4. Arsitektur Sistem

Kelarin menggunakan komunikasi **stateless berbasis JWT**.



Penjelasan Komponen

- **Client** → Akses melalui browser (HTTPS)
- **Frontend** → UI, state, routing
- **Backend** → Business logic & autentikasi
- **Database** → Penyimpanan data utama
- **Cloud Storage** → File tambahan (opsional)

5. Getting Started

Prasyarat

- Python 3.10+
- Node.js 18+
- Git CLI

Backend

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

6. Project Structure

```
cc-kelompok-ethereal_a/
├── backend/
│   ├── crud.py
│   ├── database.py
│   ├── main.py
│   ├── models.py
│   ├── requirements.txt
│   ├── schemas.py
│   └── setup.sh
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── services/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── vite.config.js
├── docs/
│   ├── api-documentation.md
│   └── setup-guide.md
├── .gitignore
└── README.md
```

Otomatisasi Lingkungan

- `setup.sh` → Setup environment otomatis (DevOps)

7. Informasi Tambahan & ERD

Entitas Utama

- **Users** → Data mahasiswa
- **Teams** → Workspace
- **Tasks** → Unit pekerjaan

Tabel Relasi

- **team_members**
 - Relasi M:M user ↔ team
 - Mengatur peran (Ketua/Anggota)
- **task_assignments**
 - Membagi tugas menjadi sub-role
 - Mengatasi pembagian kerja yang tidak jelas
 - Meningkatkan transparansi tanggung jawab

8. Roadmap Proyek

Minggu	Target	Status
1	Setup & Hello World	☒
2	REST API + Database	☒
3	React Frontend	☒
4	Full-Stack Integration	☒
5–7	Docker & Compose	☒
8	UTS Demo	☐
9–11	CI/CD Pipeline	☐
12–14	Microservices	☐
15–16	Final & UAS	☐

9. Testing & QA Report

Dokumen ini mencatat protokol pengujian, skenario, dan hasil verifikasi sistem untuk memastikan setiap modul aplikasi Kelarin berjalan sesuai spesifikasi teknis dan kebutuhan fungsional.

9.1 Log Pengujian UI

Hasil pengujian menyeluruh (10 Test Case) untuk fitur Autentikasi dan CRUD dapat dilihat pada dokumen:

[!\[\]\(758ebdf4629c903da74c2e079717ae32_img.jpg\) **Lihat Laporan Lengkap Testing UI**](#)

9.2 API Reference (Detail Endpoint)

Berikut adalah daftar lengkap endpoint yang tersedia pada aplikasi Kelarin.

Method	Endpoint	Auth	Request Body (Contoh)	Response Example (Contoh)
GET	/health	No	-	<pre>{"status": "healthy"}</pre>
GET	/team	No	-	<pre>{"team": "Ethereal"}</pre>
POST	/auth/register	No	<pre>{"email": "...", "password": "..."} </pre>	<pre>{"message": "User created"}</pre>
POST	/auth/login	No	<pre>{"email": "...", "password": "..."} </pre>	<pre>{"access_token": "eyJ..."} </pre>
GET	/auth/me	Yes	-	<pre>{"email": "user@itk.ac.id"} </pre>
GET	/tasks	Yes	-	<pre>[{"id": 1, "title": "Tugas KBT"}]</pre>
POST	/tasks	Yes	<pre>{"title": "...", "desc": "...", "due": "..."} </pre>	<pre>{"id": 2, "status": "created"}</pre>
GET	/tasks/{id}	Yes	-	<pre>{"id": 1, "title": "Tugas KBT"}</pre>
PUT	/tasks/{id}	Yes	<pre>{"title": "...", "status": "..."} </pre>	<pre>{"message": "updated"}</pre>
DELETE	/tasks/{id}	Yes	-	<pre>{"message": "deleted"}</pre>

Catatan Tambahan:

- **200 OK:** Request berhasil diproses.
- **201 Created:** Resource baru berhasil dibuat.
- **404 Not Found:** Resource tidak ditemukan (misal: ID tidak ada).
- **422 Unprocessable Entity:** Validasi data gagal (input salah).

9.3 Testing Status (Hasil Pengujian)

Berikut adalah status hasil pengujian API:

Endpoint	Test Case	Status	Catatan
/auth/register	Input user baru	☒ Pass	Berhasil (201)

Endpoint	Test Case	Status	Catatan
/auth/login	Login user valid	☒ Pass	Berhasil (200)
/tasks	Ambil data tanpa token	☒ Pass	Berhasil (401 Unauthorized)
/tasks	Ambil data dengan token	☒ Pass	Berhasil (200)

Untuk dokumentasi interaktif lengkap, jalankan server dan akses <http://localhost:8000/docs>

10. Docker & Containerization

Docker Compose adalah alat yang digunakan untuk mendefinisikan dan menjalankan aplikasi *multi-container* dengan satu file konfigurasi ([docker-compose.yml](#)).

Tujuan penggunaan Docker Compose dalam proyek ini:

1. **Konsistensi Lingkungan:** Menyamakan konfigurasi *environment* pengembangan, *testing*, dan produksi agar terhindar dari masalah "*it works on my machine*".
2. **Orkestrasi Otomatis:** Menjalankan layanan frontend, backend, dan database secara simultan dengan satu perintah.
3. **Isolasi Layanan:** Memastikan setiap komponen aplikasi berjalan di dalam *container* yang terisolasi namun tetap saling terhubung dalam satu *network* yang aman.

Docker Build & Run Instructions

Cara Menjalankan Aplikasi

Ikuti langkah-langkah berikut:

1. Pastikan Docker sudah terinstall dan sedang berjalan
2. Buka terminal di folder project ini
3. Untuk menjalankan seluruh service (frontend, backend, dan database), gunakan perintah berikut:

```
docker compose up --build
```

4. Tunggu hingga semua service berhasil berjalan
5. Buka browser dan akses:

- Frontend → <http://localhost:5173>
- Backend → <http://localhost:8000>

Jika halaman berhasil terbuka, berarti aplikasi sudah berjalan dengan baik ☒

Cara Menghentikan Aplikasi

Untuk menghentikan semua container, jalankan:

```
docker compose down
```

Cara Mengecek Status

Untuk memastikan semua service berjalan:

```
docker compose ps
```

Jika statusnya Up, berarti container berjalan dengan normal.

Cara Melihat Log

Jika terjadi error atau ingin melihat aktivitas aplikasi:

```
docker compose logs
```

Untuk melihat log secara real-time:

```
docker compose logs -f
```

Perintah Penting Docker Compose

Berikut beberapa perintah Docker Compose yang sering digunakan:

Perintah	Deskripsi
docker compose up	Menjalankan semua service (build jika diperlukan)
docker compose up -d	Menjalankan semua service di background (detached mode)
docker compose down	Menghentikan dan menghapus container serta network
docker compose down -v	Menghentikan dan menghapus container beserta volume (⚠ data akan hilang)
docker compose restart	Me-restart semua service
docker compose stop	Menghentikan container tanpa menghapus
docker compose ps	Menampilkan status semua container
docker compose logs	Menampilkan log dari semua service

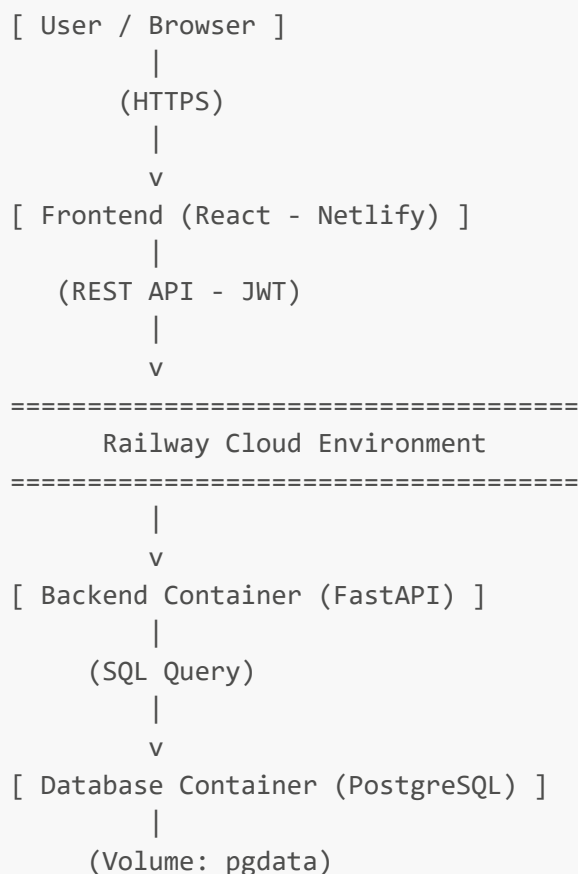
Perintah	Deskripsi
docker compose logs -f	Menampilkan log secara real-time
docker compose exec backend bash	Masuk ke dalam container backend
docker compose build	Build semua image
docker compose pull	Mengambil image dari registry

11. Deployment

- **Railway** → Backend & PostgreSQL
- **Netlify** → Frontend

11.1 Diagram Deployment (Docker & Railway)

Diagram berikut menggambarkan bagaimana aplikasi Kelarin dideploy menggunakan Docker dan dihosting pada platform Railway, serta bagaimana interaksi antar komponen sistem terjadi.



Penjelasan Deployment

- **Frontend (Netlify)**

Aplikasi frontend dibangun menggunakan React dan di-deploy sebagai static site di Netlify. User mengakses aplikasi melalui browser menggunakan protokol HTTPS.

- **Backend (Railway + Docker)**

Backend FastAPI dijalankan dalam container Docker di Railway. Service ini bertanggung jawab untuk menangani request API, autentikasi, serta logika bisnis aplikasi.

- **Database (PostgreSQL)**

Database berjalan sebagai container terpisah di Railway. Data disimpan secara persisten menggunakan volume (`pgdata`) sehingga tidak hilang meskipun container dihentikan.

- **Docker Environment**

Backend dan database berada dalam lingkungan container yang terisolasi, namun tetap dapat saling berkomunikasi melalui network internal Railway.

- **Environment Variables**

Digunakan untuk konfigurasi sistem seperti:

- koneksi database (`DB_HOST`, `DB_USER`, dll)
- JWT secret key
- port aplikasi (`PORT`)

11.2 Alur Deployment

1. User mengakses aplikasi melalui browser (Frontend di Netlify)
2. Frontend mengirim request ke backend melalui API
3. Backend menerima dan memvalidasi request (termasuk JWT)
4. Backend memproses logika aplikasi
5. Jika diperlukan, backend melakukan query ke database
6. Database mengembalikan data ke backend
7. Backend mengirim response ke frontend
8. Frontend menampilkan hasil ke user

UTS Demo Preparation Checklist

Pastikan hal-hal berikut sudah siap sebelum memulai demonstrasi aplikasi:

- ☒ **Docker Execution:** `docker compose up -d` berjalan tanpa error.
- ☒ **Frontend Accessibility:** Aplikasi dapat diakses di `localhost:5173`.
- ☒ **Backend Accessibility:** API dapat diakses dan merespons di `localhost:8000`.
- ☒ **Authentication:** Fitur Register & Login berfungsi dengan validasi yang benar.
- ☒ **Task Management (CRUD):** Fitur Create, Read, Update, Delete, pada task berfungsi.
- ☒ **Data Persistence:** Data tetap ada setelah `docker compose down` dan `up` kembali (cek volume).
- ☒ **Documentation:** `README.md` sudah mencakup instruksi Docker yang lengkap.
- ☒ **Team Readiness:** Setiap anggota memahami alur teknis (Viva Preparation).
- ☒ **Contribution:** Semua anggota tim memiliki *commit history* yang signifikan.

